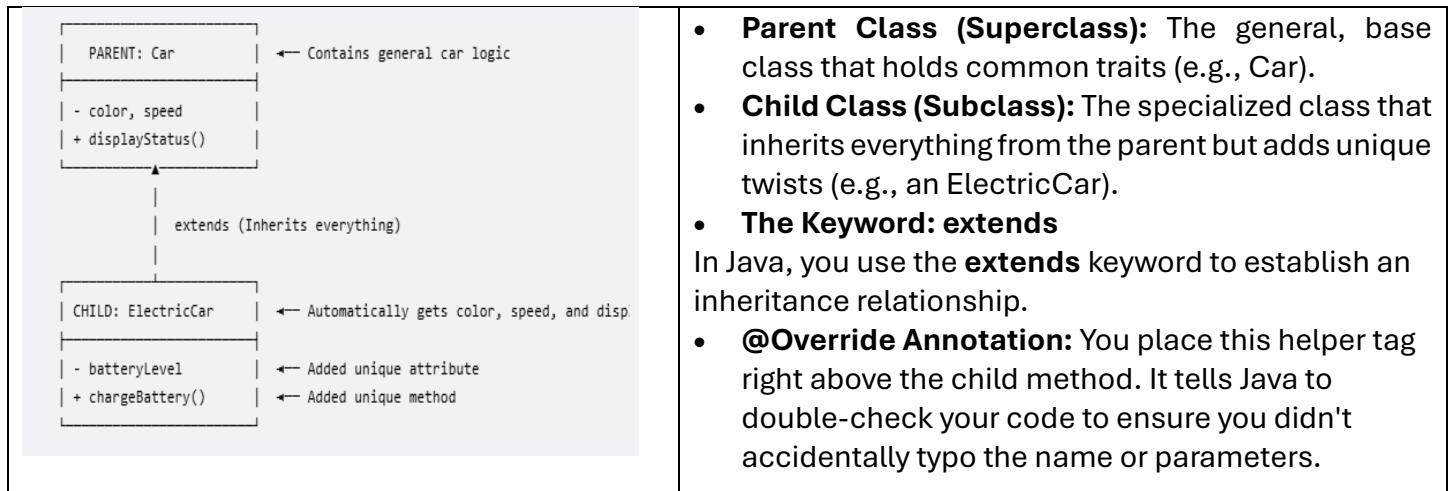


Inheritance

Inheritance is one of the pillars of Object-Oriented Programming. It allows a new class to inherit (absorb) all the attributes and methods of an existing class automatically.

Think of inheritance like **genetic traits**. A child inherits traits from their parent, but the child can also have unique traits of their own.



Why Use Inheritance?

- **Code Reusability:** You don't have to rewrite code for color, speed, or displayStatus() if you decide to build a GasCar, a Truck, or a RaceCar later. You just extend Car.
- While inheritance gives the child class all the parent's methods automatically, sometimes the default parent behavior isn't quite right for the child. **Method Overriding** occurs when a child class rewrites a method it inherited from its parent class to give it a new, specialized behavior. Child keep the exact same method name and inputs, but change what happens inside the execution brackets { }.
- ❓ **Same Name:** The method name in the child class must exactly match the parent class.
- ❓ **Same Parameters:** The inputs (arguments) must match exactly.

```

public class ElectricCar extends Car {
    public int batteryLevel;

    public ElectricCar(String color, int speed, int battery) {
        super(color, speed);
        this.batteryLevel = battery;
    }

    // — METHOD OVERRIDING HAPPENS HERE —
    @Override
    public void displayStatus() {
        // We redefine the action specifically for electric cars
        System.out.println("The " + color + " electric car is going " + speed +
            " mph with " + batteryLevel + "% battery remaining.");
    }
}
            
```

The `super` Shortcut in Overriding

Notice that inside the child's `displayStatus()`, we re-wrote the part printing the color and speed. If the parent method has a ton of complex code, re-writing it is a waste of time.

You can use `super.methodName()` to run the parent's version of the method *first*, and then add your child logic right below it: 

java

```
@Override
public void displayStatus() {
    super.displayStatus(); // Runs the parent version (Prints: The White car is going 70 mph)
    System.out.println("Battery Level: " + batteryLevel + "%"); // Adds unique child logic
}
```

```
public class Main {
    public static void main(String[] args) {

        Car basicCar = new Car("Red", 55);
        ElectricCar tesla = new ElectricCar("White", 70, 85);

        System.out.println("--- Testing Parent Method ---");
        basicCar.displayStatus();
        // Prints: The Red car is going 55 mph.

        System.out.println("\n--- Testing Overridden Child Method ---");
        tesla.displayStatus();
        // Prints: The White electric car is going 70 mph with 85% battery remaining

    }
}
```

- Java is smart under the hood. Even though both objects are used. `displayStatus()`, Java looks at the specific object type at runtime and executes the customized version for the `ElectricCar`.
- If not `Override`, Java fall back to base class function.